

Diseño de un Sistema OFDM: Transmisor y Receptor en Python (Enfoque Académico)

Introducción a la Modulación OFDM

La **Modulación por División de Frecuencia Ortogonal (OFDM)** es un esquema de modulación digital multiportadora ampliamente utilizado en comunicaciones modernas (por ejemplo, en WiFi, LTE, DVB, etc.)

1. En OFDM, el ancho de banda disponible se divide en N subcanales ortogonales, cada uno modulando un símbolo en paralelo. Es decir, en lugar de transmitir símbolos en serie sobre una sola portadora (como en modulación tradicional), OFDM transmite simultáneamente múltiples símbolos sobre diferentes subportadoras durante el mismo intervalo de tiempo 1. Cada subportadora ocupa solo una pequeña porción del espectro total y transmite a una menor velocidad de símbolo, aumentando la duración del símbolo por subportadora y ayudando a mitigar la interferencia intersímbolo (ISI) en canales dispersivos 2. La clave de OFDM es la **ortogonalidad** de las subportadoras: se seleccionan separadas por un intervalo de frecuencia $\Delta f = 1/T$ (donde T es la duración del símbolo OFDM útil, sin prefijo) para que cada sinusoidal complete un número entero de ciclos en T , asegurando integrales de correlación cero entre distintas subportadoras 3. Gracias a esta ortogonalidad, los espectros de subportadoras pueden solaparse sin interferencia entre sí, optimizando el uso de ancho de banda. Además, OFDM se implementa de forma digital: el transmisor usa una IFFT (Transformada Inversa de Fourier Rápida) para generar eficientemente la señal multicarrier y el receptor emplea la FFT para demodularla 4. De este modo, se evita tener que construir N moduladores analógicos separados; el procesamiento digital mediante FFT/IFFT permite **implementar OFDM de manera eficaz** en la práctica 4.

En las secciones siguientes se describe en detalle el diseño de un sistema OFDM baseband (banda base) con transmisor y receptor, paso a paso, incluyendo las expresiones matemáticas relevantes en cada bloque del sistema. El enfoque es académico y manual (pensado para implementación en Jupyter Notebook con Python, pero **sin incluir código**), por lo que se justifica teóricamente cada etapa del proceso. Se asume inicialmente un escenario ideal (sin ruido ni distorsión de canal) para simplificar la explicación, con miras a introducir posteriormente efectos de canal. También se sugieren parámetros razonables de diseño y se mencionan herramientas de Python útiles para la validación del sistema.

Transmisor OFDM: Proceso de Modulación Paso a Paso

El transmisor OFDM toma una secuencia binaria de entrada (bits) y la convierte en una señal modulada de múltiple subportadora lista para transmitir. Las etapas principales del modulador OFDM son:

1. **Agrupación de bits y conversión serie a paralelo**
2. **Mapeo de bits a símbolos de constelación (ej. QPSK)**
3. **Asignación de símbolos a subportadoras en el dominio de la frecuencia**
4. **Modulación multiportadora mediante IFFT (síntesis de la señal OFDM)**
5. **Inserción del prefijo cíclico (intervalo de guarda)**
6. **Conversión paralelo a serie y construcción del símbolo OFDM transmitido**

A continuación, se detalla cada bloque con sus fundamentos matemáticos correspondientes.

1. Agrupación de bits y Conversión Serie-Paralelo

El primer paso consiste en agrupar la secuencia de bits de entrada y prepararla para la modulación multicarrier. Dado un esquema de modulación digital con M símbolos posibles (por ejemplo, $M=4$ para QPSK, $M=16$ para 16-QAM, etc.), cada símbolo representa $\log_2 M$ bits. Por lo tanto, los bits de entrada se segmentan en bloques de $\log_2 M$ bits y cada bloque se asignará a un *símbolo complejo* de la constelación elegida ⁵ ⁶. Este proceso de **mapeo** (o codificación en cuadratura) produce símbolos complejos pertenecientes a una constelación en el plano I/Q (In-Phase vs. Quadrature). Por ejemplo, si usamos **QPSK** (modulación PSK cuaternaria, $M=4$), agrupamos de 2 en 2 los bits y cada par de bits (b_1, b_2) se asocia a un símbolo complejo. Una asignación típica en Gray (minimizando errores de bit por cercanía) podría ser:

- $00 \mapsto s = \frac{1+j}{\sqrt{2}}$
- $01 \mapsto s = \frac{-1+j}{\sqrt{2}}$
- $11 \mapsto s = \frac{-1-j}{\sqrt{2}}$
- $10 \mapsto s = \frac{1-j}{\sqrt{2}}$

Aquí hemos definido los símbolos QPSK como las cuatro esquinas de un cuadrado unidad (señal normalizada a potencia unitaria), pero cualquier constelación M-PSK o M-QAM podría usarse de manera similar. En general, el **mapa de bits a símbolos** consiste en interpretar $\log_2 M$ bits como un número en base 2 y asociarlo a un símbolo complejo predefinido. Matemáticamente, para QPSK podemos expresar el símbolo mapeado a partir de dos bits $b_1, b_2 \in \{0,1\}$ como:

$$X = \frac{(2b_1 - 1) + j(2b_2 - 1)}{\sqrt{2}},$$

donde $2b_1 - 1 \in \{-1, +1\}$ y $2b_2 - 1 \in \{-1, +1\}$ representan los componentes en fase (eje real) y en cuadratura (eje imaginario) respectivamente. Este sería el símbolo complejo X listo para modular una subportadora. En modulación 16-QAM (por ejemplo), se agruparían 4 bits por símbolo, y la fórmula sería análoga pero tomando valores en $\{-3, -1, +1, +3\}$ para cada cuadratura (tras normalizar).

Después del mapeo, obtenemos una secuencia de símbolos complejos **multinivel** (por ejemplo, $\dots, -1+j, 1-j, 1+j, \dots$ etc. en QPSK) que representan nuestra información en formato simbólico. A continuación, se toma un bloque de N símbolos complejos para enviarlos en paralelo, uno por subportadora, en un solo símbolo OFDM ⁷. En cada intervalo de símbolo OFDM (duración T), el transmisor agrupa N símbolos modulados; esto equivale a tomar $N \cdot \log_2 M$ bits de entrada y formarlos en N símbolos simultáneos. Para ello, se realiza una conversión **Serie-Paralelo (S/P)**: de la secuencia serial de símbolos (salida del mapeador) se toman de a N símbolos y se colocan en paralelo. Podemos denotar el conjunto de símbolos a transmitir juntos como un vector de tamaño N :

$$\mathbf{X} = [X_0, X_1, \dots, X_{N-1}],$$

donde X_k es el símbolo complejo asignado a la k -ésima subportadora en ese símbolo OFDM ⁸. Así, X_0 será el símbolo a modular la subportadora de índice 0, X_1 el de la subportadora 1, y así sucesivamente. Este vector $\mathbf{X}$ representa la **información en el dominio de la frecuencia** (cada

elemento es la “aportación” a cierta frecuencia subportadora). En la práctica, suele implementarse un *buffer* que acumula N símbolos mapeados para poder procesarlos juntos con la IFFT ⁹.

Resumen de esta etapa: se toma la secuencia binaria, se agrupan los bits según la modulación escogida produciendo símbolos complejos, y luego se conforman bloques de N símbolos (serial a paralelo) para prepararlos para la modulación OFDM multicarrier en el siguiente paso ⁷.

2. Asignación de Símbolos a Subportadoras

Una vez tenemos el vector paralelo de N símbolos $\mathbf{X} = \{X_0, X_1, \dots, X_{N-1}\}$, el siguiente paso es asociar cada símbolo a una subportadora distinta. Conceptualmente, cada X_k será la **coeficiente complejo** (amplitud y fase) con la que modularemos la k -ésima onda sinusoidal ortogonal. En un sistema OFDM ideal y básico, normalmente usamos N subportadoras para N símbolos (aunque en aplicaciones reales algunas subportadoras podrían reservarse para pilotos, control, o quedar nulas, aquí asumiremos todas son usadas para datos por simplicidad). Por tanto, se asigna X_0 a la subportadora de frecuencia f_0 , X_1 a la subportadora f_1 , ..., X_{N-1} a f_{N-1} . Las frecuencias de estas portadoras ortogonales típicamente están espaciadas por $\Delta f = 1/T$ (con T el período del símbolo OFDM útil), de modo que la k -ésima subportadora (en banda base) tiene frecuencia $f_k = k \Delta f = k/T$ para $k=0,1,\dots,N-1$. Esto garantiza la ortogonalidad en el intervalo de símbolo: $\int_0^T e^{j2\pi f_k t} e^{-j2\pi f_{k'} t} dt = 0$ para $k \neq k'$ (son sinusoides ortogonales) ³ ¹⁰. En la práctica de la implementación digital, en lugar de generar sinusoides continua, utilizaremos la IFFT para sintetizar la suma de estas subportadoras.

Desde una perspectiva de procesamiento discreto, el vector $\mathbf{X}$ de longitud N puede interpretarse como una *representación en frecuencia* (coeficientes de Fourier) de la señal OFDM a transmitir. Cada elemento X_k es la **componente espectral** en la frecuencia discreta $2\pi k/N$ radianes/muestra (equivalente a $k \Delta f$ Hz). El **modulador OFDM central es una IFFT de tamaño N** que tomará estos N símbolos en paralelo (dominio frecuencia) y los transformará a N muestras en el dominio del tiempo ¹¹. Antes de aplicar la IFFT, es importante mencionar que en algunas implementaciones se organizan los X_k en un arreglo de tamaño N colocando los símbolos de datos en ciertas posiciones y ceros en otras (por ejemplo, en OFDM real passband, se coloca la mitad superior como la conjugada de la mitad inferior para obtener señal real, y puede haber subportadoras nulas en bordes y en DC). Sin embargo, en nuestro diseño simplificado (banda base compleja), asumiremos todos los X_k son datos y que generaremos una señal compleja de banda base.

3. Modulación mediante IFFT (Síntesis de la Señal OFDM)

Llegamos al bloque fundamental: la **Transformada Inversa de Fourier Discreta (IFFT)** de tamaño N . La IFFT toma los N símbolos $X_0, \dots, X_{N-1}$ en paralelo y los convierte en N muestras temporales $x[0], x[1], \dots, x[N-1]$ que componen un símbolo OFDM en el tiempo ¹¹ ¹². Matemáticamente, las N muestras de la señal OFDM de banda base (compleja) se obtienen mediante la **IDFT** de los valores $\{X_k\}$. La ecuación de modulación (salida de la IFFT) es:

$$x[n] = \frac{1}{N} \sum_{m=0}^{N-1} X_m e^{j2\pi \frac{mn}{N}}, \quad n = 0, 1, \dots, N-1.$$

En esta expresión, N es el número de subportadoras (tamaño de la IFFT), X_m es el símbolo complejo asignado a la subportadora m , y $x[n]$ es la n -ésima muestra del símbolo OFDM en tiempo discreto. La

fórmula no es más que la definición estándar de la transformada discreta de Fourier inversa aplicada a nuestros datos modulares ¹³. La IFFT sintetiza la suma de N ondas sinusoidales ortogonales (complejas) de la forma $e^{j2\pi m n/N}$ ponderadas por los símbolos X_m . En otras palabras, estamos generando una superposición de tonos sinusoidales de diferentes frecuencias (las subportadoras), cada uno con una amplitud/fase dada por X_m ¹².

El resultado de la IFFT es la secuencia $x[0], x[1], \dots, x[N-1]$ de muestras complejas en el tiempo que constituye el símbolo OFDM compuesto, conteniendo todas las subportadoras multiplexadas ¹². Esta es la señal de OFDM en banda base para un símbolo OFDM. Su duración temporal es T (duración del símbolo útil), y típicamente la frecuencia de muestreo se elige como $f_s = N/T$ de forma que N muestras correspondan exactamente al periodo T . Nótese que si se interpreta $x[n]$ como valores de una señal continua $x(t)$ muestreada, $x(t)$ sería prácticamente:

$$x(t) = \frac{1}{N} \sum_{m=0}^{N-1} X_m e^{j2\pi f_m t}, \quad 0 \leq t < T,$$

con $f_m = m/T$ y $x[n] = x(t=n T/N)$. Esto confirma que $x[n]$ es la combinación lineal de N sinusoides ortogonales de frecuencia m/T .

Después de la IFFT, obtenemos entonces N muestras que representan el símbolo OFDM en tiempo discreto. **A continuación, realizamos la conversión Paralelo-Serial** sobre estas muestras: es decir, tomamos la secuencia $x[0] \dots x[N-1]$ y la colocamos nuevamente en serie (una tras otra) para formar la señal de tiempo continua a transmitir ¹⁴. Hasta este punto (sin considerar el prefijo cíclico aún), la señal resultante es un *símbolo OFDM baseband* de duración T y compuesto de N muestras.

Interpretación: La IFFT actúa como el modulador multiportadora: convierte los símbolos en frecuencia X_k a una señal en el tiempo $x[n]$ que físicamente equivale a modular N subportadoras a la vez y sumarlas ¹⁵. Este uso de la IFFT permite implementar OFDM eficientemente vía DSP. Cada muestra $x[n]$ es la suma de contribuciones de todos los símbolos X_m con diferentes fases rotantes; en conjunto, la secuencia temporal contiene la información de todos los subcanales.

4. Inserción del Prefijo Cíclico (Intervalo de Guarda)

Una característica esencial de OFDM es la inserción de un **intervalo de guarda** entre símbolos OFDM consecutivos para eliminar la interferencia intersímbolo (ISI) causada por el canal. El método más común de intervalo de guarda es el **Prefijo Cíclico (CP)**. Antes de transmitir el símbolo OFDM generado (las N muestras $x[0 \dots N-1]$), el transmisor toma una copia de las últimas L muestras de ese símbolo y las antepone al comienzo del mismo ¹⁶. Es decir, se añade al inicio de la señal OFDM una repetición del final del símbolo. Si denotamos L como la longitud del CP en muestras (duración $T_g = \frac{L}{N}T$ en segundos), la secuencia de salida se extiende a $N+L$ muestras: $x[N-L], \dots, x[N-1], x[0], x[1], \dots, x[N-1]$. Las primeras L muestras son el prefijo cíclico, que es idéntico a las últimas L muestras del símbolo OFDM útil.

Matemáticamente, si $x[n]$ para $0 \leq n < N$ es el símbolo OFDM, definimos el símbolo OFDM con prefijo $x_{\text{cp}}[n]$ de longitud $N+L$ como:

$$x_{\text{cp}}[n] = \begin{cases} x[n \bmod N], & n = -L, -L+1, \dots, -1, \\ x[n], & n = 0, 1, \dots, N-1, \end{cases}$$

donde hemos indexado $n = -L \dots -1$ para denotar las muestras del CP (equivalentes a $n = N-L \dots N-1$ del símbolo original). En términos simples, $x_{\text{cp}}[-L + i] = x[N-L + i]$ para $i = 0 \dots L-1$.

El prefijo cíclico tiene una duración T_g típicamente elegida mayor o igual al atraso máximo del canal (duración del impulso del canal) ¹⁷. La razón de esto es crucial: si el canal tiene dispersión multipropagación (respuesta impulsiva de longitud L_h), mientras T_g sea al menos $L_h - 1$ (en muestras), la cola del símbolo OFDM anterior no interferirá con la parte útil del siguiente símbolo (se confinará toda ISI dentro del prefijo, que luego se descartará en el receptor). Además, el CP convierte la convolución lineal del canal en una **convolución circular** efectiva sobre el símbolo OFDM ¹⁸. En otras palabras, al repetir el final del símbolo al inicio, cualquier retraso del canal hace que una versión desplazada del símbolo se alinee cíclicamente, preservando la ortogonalidad de las subportadoras. **Así, el prefijo cíclico cumple dos propósitos: (1) evita interferencia entre símbolos sucesivos (ISI) y (2) asegura ortogonalidad y no interferencia entre subportadoras (ICI) incluso en presencia de multipath, al forzar convolución circular con el canal** ¹⁸.

El costo del prefijo cíclico es una sobrecarga en tiempo (y potencia) transmitido, ya que T_g es tiempo no útil para datos (es redundancia). Suele expresarse como un porcentaje del símbolo: comúnmente $T_g = T/4$ (25%) o $T/8$ (12.5%) según el estándar y la severidad esperada del canal. Por ejemplo, en un sistema con $N=64$, un CP típico podría ser $L=16$ muestras (25%); para $N=128$, podría tomarse $L=16$ o 32 según necesidad. Esta elección es un compromiso entre robustez a ISI y eficiencia espectral.

Tras añadir el prefijo cíclico de longitud L , la duración total de un símbolo OFDM transmitido será $T_{\text{total}} = T + T_g$ ¹⁹. Durante el intervalo de guarda, la señal transmitida es simplemente una repetición del final del símbolo OFDM, por lo que no contiene nueva información pero permite al receptor "absorber" los desvanecimientos del canal sin corromper el símbolo siguiente.

Finalmente, la señal en tiempo discreto (el símbolo OFDM más su prefijo) se convierte a señal analógica mediante un DAC (convertidor digital-analógico) y se modula a la frecuencia de radio (RF) deseada para la transmisión por el canal inalámbrico ²⁰. En un entorno de simulación baseband (como Python), usualmente modelamos directamente la señal complex-baseband y omitimos la conversión a RF, pero conceptualmente: el transmisor OFDM completo realiza:

- **Codificación de canal** (si existiese, ej. FEC, aquí opcional),
- **Mapeo** de bits a símbolos complejos (modulación digital M-PSK/QAM),
- **Conversión serie a paralelo** formando vectores de N símbolos,
- **IFFT** de tamaño N (modulación multiportadora digital),
- **Inserción del prefijo cíclico** (intervalo de guarda),
- **Conversión paralelo a serie** de las muestras y **conversión a señal analógica** (DAC) seguida de **upconversion a RF** ²¹.

En resumen, el **transmisor OFDM** toma bits y produce una señal de banda base modulada con múltiples subportadoras ortogonales. Todo este flujo se puede implementar de forma eficiente con herramientas digitales (FFT/IFFT), lo cual ha sido clave para la adopción de OFDM a gran escala ²².

Receptor OFDM: Proceso de Demodulación Ideal

El receptor OFDM realiza las operaciones inversas del transmisor para recuperar los datos originales. En un escenario ideal (canal sin distorsión y sin ruido), el objetivo es demostrar que la señal se puede demodular exactamente, recuperando los símbolos x_k transmitidos en cada subportadora, y de ahí los bits transmitidos. A continuación se describen los pasos del receptor OFDM ideal:

1. **Recepción RF y conversión a baseband discreto**
2. **Detección de inicio de símbolo y sincronización (supuesto logrados)**
3. **Remoción del prefijo cíclico**
4. **Transformada FFT para obtener las subportadoras**
5. **Demapeo de subportadoras: detección de símbolos (criterio de distancia mínima)**
6. **Conversión paralelo a serie y mapeo inverso a bits**
7. **Cálculo de BER (para evaluación de desempeño)**

Desarrollemos cada etapa asumiendo inicialmente un canal ideal y sincronización perfecta:

1. Front-end Analógico y Conversión A/D (Recepción)

En la práctica, la señal OFDM transmitida viaja por el canal (inalámbrico o cableado) y llega al receptor como una señal analógica bandapass (en RF) posiblemente atenuada, distorsionada y con ruido. El receptor primero lleva esta señal a **banda base** mediante un mezclador con la portadora (demodulación radioeléctrica), y luego la convierte a digital usando un convertidor ADC ²³. Este proceso proporciona una secuencia de muestras complejas que representan la señal OFDM recibida en banda base. Se asume que se ha logrado la **sincronización de tiempo y frecuencia**, es decir, que el receptor sabe dónde comienza el símbolo OFDM en la secuencia de muestras y que se ha corregido cualquier desfase de frecuencia de portadora (estas tareas de sincronización y estimación de canal son críticas pero quedan fuera del alcance inmediato de este análisis idealizado). Bajo esta suposición, el receptor puede tomar exactamente los bloques de muestras correspondientes a cada símbolo OFDM.

Nota: En nuestro escenario ideal de simulación, podemos alinear perfectamente los símbolos transmitidos y recibidos, ignorando problemas de sincronización. Asimismo, inicialmente consideramos un canal ideal (respuesta impulsiva delta) y ausencia de ruido, de modo que las muestras recibidas sean esencialmente iguales a las transmitidas (después del ADC). Más adelante se puede incorporar un canal selectivo y ruido AWGN para evaluar la robustez.

2. Remoción del Prefijo Cíclico

Una vez el receptor tiene las muestras digitales correspondientes a un símbolo OFDM (de duración $T_{\text{total}} = T_g + T$), el primer paso de la demodulación es remover el prefijo cíclico. Recordemos que el prefijo cíclico son las primeras L muestras, que consisten en una copia redundante. El receptor **identifica y descarta estas L muestras iniciales** (el intervalo de guarda) ²⁴. Al hacerlo, se elimina cualquier interferencia del símbolo anterior que pudiera haber entrado en el intervalo de guarda (suponiendo T_g adecuado, no debería haber interferencia en la parte útil). Tras quitar el CP, quedan exactamente N muestras que corresponden al símbolo OFDM limpio en el dominio del tiempo: $y[0]$, $y[1]$, ..., $y[N-1]$. En un caso ideal sin canal ni ruido, estas muestras $y[n]$ deben coincidir con las originales $x[n]$ transmitidas (quizás con algún retardo global o fase común, pero equivalentes).

La razón de descartar el CP es precisamente que convierte la **convolución lineal en circular**: cualquier señal multipropagada fuera del intervalo de guarda ya no afecta el símbolo útil. Así, al iniciar la FFT en la muestra $y[0]$ (que corresponde en tiempo al comienzo real del símbolo OFDM transmitido), nos aseguramos de procesar un bloque de N muestras que constituye un período de la convolución circular entre $x[n]$ y la respuesta del canal. En el caso ideal (canal delta), $y[n] = x[n]$, por lo que la circularidad no es un tema, pero es importante en general. En resumen, tras remover el prefijo, el receptor tiene el símbolo OFDM de longitud N listo para ser transformado al dominio de la frecuencia, *sin* interferencia intersimbólica ²⁴.

3. Transformada FFT: Recuperación de las Subportadoras

El siguiente paso clave es aplicar la **Transformada Discreta de Fourier (FFT)** de tamaño N a las N muestras recibidas $y[0..N-1]$. La FFT convierte la señal de vuelta al dominio de la frecuencia, produciendo N valores complejos que representan las componentes en cada subportadora ²⁵. En fórmula, la FFT calcula (según la convención estándar sin factor $1/N$):

$$Y[k] = \sum_{n=0}^{N-1} y[n] e^{-j2\pi \frac{k n}{N}}, \quad k = 0, 1, \dots, N-1.$$

Si el sistema está perfectamente sincronizado y el canal es ideal, la relación entre $Y[k]$ y los símbolos transmitidos X_k es directa. Recordemos que en el transmisor hicimos $x[n] = \frac{1}{N} \sum_m X_m e^{j2\pi m n/N}$. Aplicando la FFT en el receptor:

$$Y[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi k n/N} = \sum_{n=0}^{N-1} \frac{1}{N} \sum_{m=0}^{N-1} X_m e^{j2\pi m n/N} e^{-j2\pi k n/N}.$$

Intercambiando sumas y utilizando la ortogonalidad de las exponenciales complejas (sumas geométricas), todos los términos con $m \neq k$ se cancelan y queda $Y[k] = X_k$ (exactamente, porque incluimos $1/N$ en la IFFT) **en ausencia de ruido y distorsión**. Por lo tanto, idealmente la FFT recupera los símbolos transmitidos $X_0, X_1, \dots, X_{N-1}$ en cada subcanal ²⁶. En la práctica, si hubo un canal con respuesta $h[n]$ y ruido, el resultado de la FFT será $Y[k] \approx H_k \cdot X_k + Z_k$, donde H_k es la respuesta en frecuencia del canal en la k -ésima subportadora y Z_k es el ruido en esa subportadora ²⁷ ²⁸. Pero para el caso ideal $H_k=1$ y $Z_k=0$, obtenemos $Y[k] = X_k$.

Este es el **gran atractivo de OFDM**: con CP suficientemente largo, cada subportadora se comporta como un canal plano independiente en frecuencia (sin interferencia de otras subportadoras ni de otros símbolos) ²⁹. Así, la ecualización (compensación del canal) en OFDM es trivial: bastaría dividir $Y[k]$ por H_k en caso de desvanecimientos, subportadora por subportadora. En nuestro receptor ideal no hace falta ecualizar (asumimos $H_k=1$).

Tras la FFT, obtenemos el conjunto de valores $\tilde{X}_k = Y[k]$ que, idealmente, deberían coincidir con los símbolos originalmente transmitidos X_k . Ahora el problema se reduce a detectar qué símbolo de la constelación fue enviado en cada subportadora.

4. Demapeo de Subportadoras y Detección de Símbolos

Los N valores complejos $\tilde{X}_0, \tilde{X}_1, \dots, \tilde{X}_{N-1}$ resultantes de la FFT corresponden a las subportadoras en el dominio de frecuencia. En un sistema práctico, puede haber símbolos de referencia

(pilotos) conocidos insertados en algunas subportadoras para estimar el canal, pero en nuestro caso asumiremos todos son datos desconocidos que hay que detectar. El proceso de **demapeo o detección de símbolos** consiste en, para cada $\tilde{x}_k$, decidir cuál símbolo x_k de la constelación digital se había transmitido en esa subportadora ³⁰.

Bajo condiciones ideales (sin ruido), $\tilde{x}_k$ será exactamente igual a uno de los símbolos de la constelación (quizá con una fase común rotada, si hubiera un desfase global, lo cual se corrige con sincronización). Sin embargo, con ruido, $\tilde{x}_k$ puede estar desplazado del punto ideal. El criterio óptimo de detección para constelaciones equiprobables en ausencia de información de fase es el de **máxima verosimilitud**, que equivale a elegir el **punto de la constelación más cercano en distancia Euclídea** al valor recibido $\tilde{x}_k$. En términos sencillos, se compara $\tilde{x}_k$ con todas las posibles símpolos de la constelación (ej. los 4 puntos QPSK conocidos) y se elige aquel cuya diferencia $|\tilde{x}_k - S|$ sea mínima. Este proceso se hace para cada subportadora independientemente, produciendo decisiones $\hat{x}_0, \hat{x}_1, \dots, \hat{x}_N$ en QPSK). Entonces matemáticamente la detección es:}. Denotemos $\mathcal{C}$ al conjunto de símbolos de la constelación (por ejemplo, $\mathcal{C} = \{\frac{1+j}{\sqrt{2}}, \frac{-1+j}{\sqrt{2}}, \frac{-1-j}{\sqrt{2}}, \frac{1-j}{\sqrt{2}}\}$

$$\hat{x}_k = \arg\min_k | \tilde{x}_k - S |$$

que entrega el símbolo S de la constelación más cercano a $\tilde{x}_k$. En el caso ideal sin ruido, $\tilde{x}_k$ ya es exactamente un símbolo válido, por lo que la decisión es correcta con probabilidad 1.

Tras esta detección por distancia mínima, obtenemos un conjunto de símbolos detectados $\{\hat{x}_k\}$, uno por subportadora. Estos deberían corresponder (en el caso ideal) exactamente a los símbolos originales $\{x_k\}$ transmitidos.

5. Conversión Paralelo-Serie y Reconstrucción de Bits

El último paso es recuperar la secuencia de bits a partir de los símbolos detectados $\hat{x}_k$. Dado que en el transmisor cada símbolo representaba $\log_2 M$ bits según el mapeo de constelación, el receptor simplemente aplica el **mapeo inverso**: convierte cada símbolo detectado al grupo de bits correspondiente ³⁰. Por ejemplo, si $\hat{x}_k = \frac{1+j}{\sqrt{2}}$ en nuestro esquema QPSK Gray mencionado, se traduciría a bits 00. Este proceso se realiza para cada subportadora obteniendo $N \cdot \log_2 M$ bits por símbolo OFDM. A continuación, el receptor recompone la secuencia binaria original ensamblando los bits de todos los subcanales. Esto implica una conversión **Paralelo a Serie (P/S)**: los N símbolos de un bloque (ya convertidos a bits) se concatenan en serie para reconstruir el bloque de bits transmitido en ese intervalo ³⁰. Repetido para cada símbolo OFDM en el tiempo, se arma la secuencia completa de bits recuperados.

En un sistema ideal sin errores, la secuencia de bits obtenida será idéntica a la original transmitida.

6. Cálculo de la Tasa de Error de Bits (BER)

Para evaluar el desempeño de la comunicación, se suele calcular la **Bit Error Rate (BER)**, que es la fracción de bits erróneos en la recepción respecto al total de bits transmitidos. Si denotamos N_b el número total de bits transmitidos y N_e el número de bits detectados incorrectamente, entonces:

$$\text{BER} = \frac{N_e}{N_b}$$

En la primera etapa de pruebas de un sistema OFDM (canal ideal, sin ruido), esperamos $\text{BER} = 0$ exactamente ³¹, ya que no debería haber ningún error: los bits transmitidos se recuperan idénticos en el receptor. Este escenario de $\text{BER} = 0$ es un punto de partida importante para validar la implementación del transmisor y receptor OFDM ³¹. Posteriormente, al introducir degradaciones (ruido AWGN, desvanecimientos, etc.), aparecerán errores y la BER será mayor que 0; entonces se analiza el desempeño bajo esas condiciones. Por ejemplo, en presencia solo de ruido AWGN, cada subportadora sufre errores conforme a la curva teórica de BER de la modulación digital usada (ej. QPSK en AWGN) ³² ³³, con quizás pequeñas pérdidas por el overhead del CP. En canales con multipath, OFDM muestra su ventaja de poder ecualizar fácil subportadora a subportadora y usar técnicas de codificación para mitigar fades selectivos ³⁴ ³⁵. Pero en todos los casos, la BER se calcula comparando los bits transmitidos con los recibidos después de la detección.

Para nuestros fines académicos iniciales, verificaremos que con canal ideal y sin ruido la BER calculada es 0 (es decir, $N_e=0$ errores) confirmando el correcto funcionamiento de la cadena OFDM.

Justificación Teórica de Cada Paso

Cada etapa descrita del sistema OFDM tiene fundamentos teóricos sólidos en las comunicaciones digitales multicarrier. A modo de **justificación resumida de cada paso**:

- **Mapeo de bits a símbolos (Modulación digital):** Permite convertir datos binarios en símbolos adecuados para la transmisión. Usar constelaciones PSK/QAM es eficiente y permite aprovechar técnicas conocidas de modulación. La agrupación de bits incrementa la tasa de datos por símbolo. Este paso es estándar en cualquier sistema digital, OFDM simplemente lo aplica N veces en paralelo. (Referencias: modulaciones M-PSK/QAM en textos de comunicaciones digitales).
- **Serial-Paralelo hacia N subportadoras:** La transmisión en paralelo de N símbolos reduce la velocidad de símbolo en cada subcanal, incrementando la duración simbólica y mitigando ISI en canales dispersivos ³⁶. Se justifica porque un canal multipath de dispersión dada causa menos ISI cuanto más largo es el símbolo. OFDM logra símbolos largos dividiendo el flujo en paralelo. Además, esta segmentación habilita el uso de la IFFT para multiplexar las portadoras de forma eficiente ⁴.
- **IFFT (IDFT como modulador):** Es el núcleo matemático de OFDM. Justificación: generar N modulaciones sinusoidales ortogonales individualmente sería costoso, pero la IFFT las genera simultáneamente en $O(N \log N)$ operaciones. La teoría de Fourier garantiza que la IDFT exactamente construye la suma de senoides ortogonales con las amplitudes dadas por X_k . Así, la IFFT actúa como un banco de moduladores perfectamente sincronizados ²² ¹⁵. Se apoya en la ortogonalidad: $e^{j2\pi m n/N}$ para m diferentes son base ortogonal, por lo que la IFFT y la FFT son inversas perfectas bajo condiciones ideales.
- **Prefijo Cíclico:** Justificación: en transmisión con convolución de canal, el CP de longitud $\geq L_h - 1$ convierte la convolución lineal en *circular*, lo cual en frecuencia se traduce en multiplicación por la respuesta en frecuencia H_k de cada subportadora (diagonalización del canal) ³⁷ ²⁹. El CP evita ISI asegurando que cada símbolo OFDM recibo esté libre de interferencia del anterior ¹⁸. Es un

truco inteligente para mantener ortogonalidad en canales selectivos, a costa de cierta redundancia temporal.

- **Remoción del CP en el receptor:** Simple pero esencial: eliminando las muestras redundantes iniciales se restituye el bloque de longitud N que corresponde a un período de la convolución circular. Esto garantiza que aplicar la FFT recupere subportadoras limpias sin mezcla entre símbolos ²⁴. La condición es que la sincronización sea correcta para cortar exactamente el CP y empezar en el símbolo.
- **FFT en el receptor:** Justificación: dado que la IFFT moduló los datos, la FFT demodula. La FFT separa las componentes ortogonales y devuelve los símbolos X_k (afectados por el canal). Matemáticamente, la FFT es la inversa de la IFFT (salvo normalizaciones), por lo que en ausencia de error es lógico que recupere X_k . En presencia de canal, la FFT produce $\tilde{X}_k = H_k X_k + Z_k$, que sigue siendo más fácil de manejar (ecualización por ganancia compleja) que una convolución en el tiempo ³⁸ ³⁹.
- **Detección de símbolos (Demapping):** Se basa en teoría de decisión para modulación digital: con ruido AWGN, la decisión ML equivale a elegir la constelación más cercana. Este criterio minimiza la probabilidad de error de símbolo. En canal ideal sin ruido, siempre acierta. El demapeo simplemente revierte el mapa de bits a símbolos aplicado en el transmisor, un proceso uno-a-uno.
- **Reconstrucción de bits:** Inversa directa del agrupamiento inicial. Dado que no se pierde información (cada símbolo decodificado mapea a exactamente un patrón de bits), se recupera la secuencia original (con posibles errores si algún símbolo se detectó mal). La BER cuantifica esos errores comparando bit a bit.

Cada uno de estos pasos está respaldado por abundante literatura. OFDM, desde sus orígenes (Weinstein y Ebert, 1971 usando DFT) ²², ha sido analizado en profundidad: por ejemplo, las referencias clásicas destacan la importancia de la ortogonalidad y el CP ⁴⁰ ⁴¹, así como el uso de modulación multiportadora para combatir ISI ⁴². Nuestro diseño sigue estos principios teóricos paso a paso.

Parámetros de Diseño y Pruebas Recomendados

Al implementar y probar un sistema OFDM a nivel académico (por ejemplo, en Python), es útil escoger parámetros manejables pero representativos. Algunas sugerencias razonables son:

- **Número de subportadoras (N):** Valores pequeños como $N = 64$ o $N = 128$ son recomendables para pruebas iniciales. Por ejemplo, $N=64$ es utilizado en el estándar WiFi 802.11a (52 subportadoras útiles y algunas pilotos, pero 64 en la IFFT) ⁴³. Estos tamaños permiten observar el comportamiento OFDM manteniendo la complejidad baja. También son potencias de 2, optimizando la FFT.
- **Modulación por subportadora:** Comenzar con **QPSK ($M=4$)** facilita la visualización de constelaciones y es robusta. Se pueden también probar esquemas como 16-QAM ($M=16$) para ver el efecto de modulación de orden superior en BER. Inicialmente, usar la misma modulación en todas las subportadoras.

- **Prefijo cíclico (\$L\$):** Elegir un prefijo de, por ejemplo, $L = N/4$ (25% del símbolo) o $N/8$ (12.5%), según experimentos. Con $N=64$, $L=16$ es una opción típica; con $N=128$, $L=16$ o 32 . Un CP del 25% es generoso y asegura ortogonalidad incluso con canales relativamente largos, a costa de algo de eficiencia. Uno del 12.5% es más eficiente pero requiere canal menos dispersivo. Estas opciones permiten comprobar la ausencia de ISI cuando L es suficiente y la aparición de ISI si L se queda corto deliberadamente.
- **Cantidad de símbolos OFDM simulados:** Para obtener estadísticas (por ejemplo, calcular BER bajo ruido), puede enviarse un número grande de bits. Por ejemplo, 10^5 bits transmitidos (~1563 símbolos OFDM de 64 subportadoras a 2 bits/subportadora con QPSK) para tener muestras suficientes en la evaluación de BER. En tests determinísticos (sin ruido, canal ideal) basta con unos pocos símbolos para verificar la reconstrucción exacta.
- **Canal y SNR:** Empezar con **canal ideal** (respuesta impulsiva $\delta(t)$) y **sin ruido** para debug. Luego introducir **ruido AWGN** progresivamente para ver cómo la nube de puntos de constelación se dispersa y la BER deja de ser 0, siguiendo la curva teórica de QPSK en AWGN [44] [45]. También se puede modelar un **canal multipath simple** (ej. dos retardos) para verificar que sin CP hay ICI/ISI y con CP la ortogonalidad se mantiene. Variar la relación T_g vs. dispersión del canal en experimentos.
- **Tiempo de símbolo y espaciado:** Al simular, se puede fijar $T=1$ (unidad) y así $\Delta f = 1$ Hz en unidades normalizadas, o cualquier valor conveniente. Lo importante es mantener $\Delta f = 1/T$ para ortogonalidad. Con frecuencia de muestreo N/T , el tono k corresponde a k ciclos en la ventana. En Python, usar directamente los índices de IFFT asocia correctamente las frecuencias.

Estas configuraciones permiten explorar el funcionamiento de OFDM de forma controlada. Por ejemplo, un caso práctico: $N=64$, $CP=16$, QPSK, canal AWGN con $E_b/N_0 = 10$ dB. Se puede medir la BER y comparar con la predicha teóricamente para QPSK. Luego quizá probar 16-QAM y ver su BER vs SNR comparada. También visualizar el espectro OFDM (mediante FFT de la señal transmitida) para verificar que se ocupan N sub-bandas como se espera, o la gráfica de la PAPR (Peak-to-Average Power Ratio) de la señal transmitida para discutir otra característica de OFDM.

Herramientas de Python para Implementación y Análisis

Aunque no incluiremos código en este informe, vale la pena mencionar las herramientas de Python que facilitan la implementación de un sistema OFDM y su análisis:

- **NumPy:** Biblioteca fundamental para el manejo de arreglos y cálculos numéricos. Se puede usar `numpy.random` para generar bits aleatorios y también las funciones de transformada rápida de Fourier. En particular, `numpy.fft.ifft` y `numpy.fft.fft` serán empleadas para implementar la IFFT del transmisor y la FFT del receptor, respectivamente, aprovechando su eficiencia. NumPy maneja números complejos de forma nativa, lo cual es crucial para OFDM (símbolos y señales complejas).
- **SciPy:** Ofrece funcionalidades adicionales; por ejemplo `scipy.fftpack` como alternativa para FFT/IFFT, o `scipy.signal` para generar respuestas al impulso de canales multipath o filtrar

señales si se quisiera modelar el canal de forma más detallada. También puede utilizarse `numpy.convolve` o `scipy.signal.lfilter` para aplicar la convolución señal-canal si se simula un canal FIR. Para generar ruido AWGN, se puede usar `numpy.random.normal` para parte real e imaginaria con la potencia adecuada.

- **Matplotlib:** Biblioteca de visualización muy útil para **graficar resultados**. Por ejemplo, se puede graficar la constelación recibida (`plt.scatter(np.real(Y_k), np.imag(Y_k))`) para observar los puntos alrededor de los ideales, o graficar la evolución de BER vs SNR. También sirve para ilustrar la forma de onda temporal de un símbolo OFDM (mostrando quizá que parece ruido pseudoruido), o su espectro (mediante una FFT de la señal con `plt.plot(np.fft.fftfreq, 20*np.log10(|FFT|))`) para ver el espectro plano ocupando N subcanales). Las gráficas ayudan a entender conceptos como ortogonalidad (espectros sinc superpuestos), interferencia cuando falta CP, etc.
- **Jupyter Notebook:** Como entorno, es ideal para documentar paso a paso el proceso (texto, fórmulas, código y gráficos intercalados). Permite recalcular rápidamente al cambiar parámetros e ir validando cada bloque (por ejemplo: verificar que la FFT recupera símbolos transmitidos exactamente en el caso ideal, imprimir los valores X_k vs Y_k para comprobar $Y_k \approx X_k$).
- **Cálculo de BER:** Puede hacerse de forma vectorizada comparando arrays de bits. En Python sería algo como `np.mean(bits_tx != bits_rx)` para obtener la fracción de bits distintos (eso da la BER). Esta simple instrucción evita loops y aprovecha operaciones rápidas de NumPy. Igualmente, se pueden insertar contadores para símbolos erróneos, etc., pero a nivel conceptual la comparación directa bit a bit es suficiente.
- **Otras utilidades:** Si se desean generar constelaciones Gray automáticamente, bibliotecas como `itertools` pueden ayudar a crear tablas de mapeo. Sin embargo, para propósitos académicos, codificar manualmente el mapeo QPSK/16QAM es instructivo. Python también puede facilitar el cálculo de métricas como PAPR (ej. `np.max(|x|^2)/np.mean(|x|^2)`) para discutir potencia pico, o autocorrelaciones para detectar inicio de símbolo (aunque eso es más avanzado, algunas implementaciones usan correlaciones para sincronización).

En resumen, Python (con NumPy/Matplotlib) proporciona todas las herramientas necesarias para **armar un transmisor y receptor OFDM "a mano"** y validar cada paso con cálculos y visualizaciones. La implementación manual en un Notebook, siguiendo el flujo descrito arriba, es altamente didáctica: permite ver los datos transformarse desde bits hasta onda modulada y viceversa, y comprobar empíricamente las propiedades teóricas (ortogonalidad, efecto del CP, BER, etc.).

Conclusión: Este informe ha descrito detalladamente el diseño de un sistema OFDM digital, explicando cada bloque del transmisor y receptor con fundamento matemático. Se ha demostrado cómo, mediante la IFFT/FFT y el prefijo cíclico, OFDM logra transmitir en paralelo múltiples datos de forma ortogonal y resistente a ISI. Con las bases aquí expuestas, se está en condiciones de proceder a una implementación práctica en Python, sabiendo qué se debe hacer en cada etapa y por qué. En un Notebook, se puede codificar cada bloque (siguiendo el orden presentado), usar las herramientas mencionadas para verificar resultados, y así consolidar la comprensión profunda de OFDM en un contexto académico y experimental.

Referencias Utilizadas:

- Documentos académicos sobre OFDM y ortogonalidad 1 6 15
- Notas e informes técnicos de OFDM (multiportadora, prefijo cíclico, etc.) 12 18 25
- Artículos y publicaciones sobre desempeño de OFDM en canal AWGN y selectivo 32 34 .

1 2 3 4 5 7 8 11 12 13 14 16 17 18 19 20 21 23 24 25 26 27 28 29 30 31 32 33 34 35

36 37 38 39 44 45 Investigación Detallada sobre la Modulación OFDM.pdf

file:///file-HQEzrtNUBaMvREpDPZ4KeB

6 9 10 15 22 40 41 42 Dialnet ModulacionMultiportadoraOFDM 4797263 | PDF | Multiplexación por división de frecuencia ortogonal | Modulación

<https://es.scribd.com/document/367679759/Dialnet-ModulacionMultiportadoraOFDM-4797263>

43 OFDM and OFDMA Subcarriers - What Are the Differences?

<https://www.extremenetworks.com/resources/blogs/ofdm-and-ofdma-subcarriers-what-are-the-differences>